

Binary Reverse Engineering Practice Portfolio

Bomb Lab & Validation Binary Analysis

김윤아

성균관대학교 소프트웨어학과 3 학년

악성코드 분석 직무 지원용 포트폴리오

본 포트폴리오는 시스템프로그래밍 수업에서 수행한 바이너리 분석 실습을 바탕으로, 악성코드 분석 직무와 연결되는 리버싱 경험을 정리한 문서입니다.

지원 분야	악성코드 분석 / 바이너리 리버싱
핵심 경험	Bomb Lab phase 분석, 입력 검증 루틴 역분석, XOR/shift/table mapping 분석
사용 도구	GDB, objdump/disassembly, x86-64 Assembly, Linux CLI
관련 수업	시스템프로그래밍, 컴퓨터구조, 운영체제
관심 역량	정적 분석, 동적 분석, 실행 흐름 추적, 문자열/난독화 로직 해석

1. 프로젝트 개요

본 포트폴리오는 두 개의 수업 실습 보고서를 채용 포트폴리오 형식으로 재가공한 것입니다.

- Bomb Lab: 여러 phase 의 어셈블리 코드를 분석하고, 입력 검증 조건을 만족하는 값을 역추적한 실습
- Custom Validation Binary: student ID 와 password 검증 루틴을 분석하고, XOR/byte reorder 기반 변환 로직을 역산한 실습
- 직무 연결성: 악성코드 분석에서 필요한 "컴파일된 실행 파일의 동작을 코드 없이 파악하는 능력"과 직접 연결됨

2. 기술 스택 및 분석 관점

분석 영역	수행 내용	악성코드 분석과의 연결
Disassembly 해석	x86-64 명령어, cmp/jmp/call/ret 흐름 추적	정적 분석에서 함수 역할과 실행 흐름 파악
레지스터/스택 관찰	GDB 로 rax, rdx, rsp 기반 입력값 저장 위치 확인	동적 분석에서 조건 충족 여부와 메모리 상태 확인
문자열/테이블 분석	문자열 상수, 배열, lookup table 기반 변환 분석	난독화된 문자열 또는 C2/설정값 추출의 기초
비트 연산 역산	xor, and, shift, byte reorder 로직 역추적	패킹/간단한 난독화/검증 루틴 해석의 기초
자료구조 추적	linked list node 값과 포인터 재배열 분석	메모리 구조 기반 행위 분석 능력 강화

3. Case Study - func4 재귀 함수 기반 입력 검증 분석

Bomb Lab phase_4 에서 재귀 함수가 반환하는 값을 기준으로 입력을 검증하는 구조를 분석한 사례입니다.

항목	정리 내용
분석 목표	입력값이 func4 내부 재귀 호출을 거친 뒤 목표 반환값 89 를 만들도록 하는 조건을 파악
사용 관점	scanf 입력 위치 확인 -> 첫 번째 입력 범위 확인 -> func4 호출 전 인자 확인 -> 반환값 eax 비교
직접 관찰 포인트	breakpoint, stepi, info registers 를 사용해 rdi/rsi/rdx 와 eax 의 변화를 확인
직무 연결	악성코드가 특정 조건에서만 실행되도록 숨겨둔 trigger condition 또는 environment check 분석과 유사

분석 흐름

- 입력 포맷 문자열을 확인해 정수 2 개를 입력받는 구조임을 확인했습니다.
- phase_4 에서는 첫 번째 입력값의 범위를 제한하고, func4 반환값이 특정 값과 일치하는지 비교합니다.
- func4 내부는 단순 산술식이 아니라 재귀 호출을 통해 결과값을 누적하는 구조이므로, 반환값 eax 가 목표값이 되는 경로를 추적했습니다.
- 두 번째 입력값 역시 반환값과 비교되므로, 최종적으로 "함수 결과값"과 "사용자 입력값"이 동시에 조건을 만족해야 합니다.

```
# 실제 분석 때 사용한 GDB 흐름 예시
(gdb) disas phase_4
```

```
(gdb) x/s 0x555555558122      # 입력 포맷 확인: "%d %d"
(gdb) break func4
(gdb) run
(gdb) stepi
(gdb) info registers rdi rsi rdx rax eax

# 관찰 기준
# - func4 호출 직전 인자 값
# - 재귀 호출 후 eax 반환값
# - phase_4 복귀 후 eax == 89 비교 여부
```

정리한 의사코드

```
read two integers: input1, input2
if input_count != 2: explode()
if input1 > 21: explode()

result = func4(input1, ...)
if result != 89: explode()
if input2 != 89: explode()
```

재귀 함수의 인자, 반환값, 비교 조건을 추적해 검증 로직을 복원했다

4. Case Study - XOR 및 Lookup Table 기반 문자열 변환 분석

입력 문자열이 내부 테이블과 비트 연산을 거쳐 목표 문자열로 변환되는 구조를 역산한 사례입니다.

항목	정리 내용
분석 목표	사용자 입력 문자열이 목표 문자열로 변환되는 규칙을 파악하고 역방향으로 입력 후보를 계산
핵심 연산	각 문자에 XOR 5 적용, AND 0xf 적용, 16 개 문자 테이블에서 인덱싱
관찰한 테이블	yosleidutranmxbz 형태의 lookup table
직무 연결	악성코드에서 자주 보이는 간단한 문자열 은닉/복호화 루틴 분석의 기초

분석 흐름

- 문자열 길이 조건을 먼저 확인해 정확히 6 글자를 요구한다는 점을 파악했습니다.
- 메모리의 16 바이트 테이블을 확인하고, 입력 문자가 바로 비교되는 것이 아니라 테이블 인덱스로 변환됨을 확인했습니다.
- 입력 문자마다 XOR 와 AND 가 적용되므로, 출력 문자열을 기준으로 가능한 입력 문자를 역산했습니다.
- 목표 문자열이 직접 입력값이 아니라 변환 결과라는 점이 핵심입니다.

```
# GDB 관찰 예시
(gdb) x/s 0x55555555817f      # "%d %s"
(gdb) x/16c 0x555555558590    # lookup table 확인

table = [ y, o, s, l, e, i, d, u, t, r, a, n, m, x, b, z ]

for each character c in input_string:
    index = (c ^ 0x5) & 0xf
    decoded_char = table[index]

if decoded_string == "sunday": pass
```

값 흐름표

단계	값/연산	의미	분석 포인트
1	input char	사용자 입력 한 글자	문자열이 직접 비교되지 않음
2	c ^ 0x5	하위 비트 변환	간단한 난독화/변환
3	& 0xf	0~15 인덱스로 제한	테이블 접근 범위 생성
4	table[index]	출력 문자 생성	목표 문자열과 비교
문자열이 평문으로 바로 보이지 않을 때, 어떤 연산을 통해 생성되는지 추적했다			

5. Case Study - Linked List 재배열 및 정렬 조건 분석

입력값이 노드 선택 순서로 변환되고, 최종 linked list 가 특정 정렬 조건을 만족해야 하는 구조를 분석한 사례입니다.

항목	정리 내용
분석 목표	입력 배열이 어떤 방식으로 node 선택 순서로 변환되는지 파악하고, 최종 비교 조건을 만족하는 순서를 도출
핵심 구조	입력 범위/중복 검사 -> 9-A[i] 변환 -> node pointer 선택 -> node value 비교
분석 대상	node1~node8 의 value 와 next pointer 흐름
직무 연결	실행 중 메모리의 자료구조와 포인터 이동을 복원하는 훈련

분석 흐름

- 먼저 입력값이 1~8 범위이며 서로 중복되지 않아야 하는 조건을 확인했습니다.
- 검증 이후 입력값이 9-A[i] 형태로 변환되어 실제 node 선택 인덱스로 사용됩니다.
- 선택된 node 들의 주소가 스택에 저장되고, 이후 next pointer 를 재연결하는 흐름을 확인했습니다.
- 마지막 비교 루프에서 앞 node 값과 뒤 node 값을 비교하므로, node value 가 요구하는 정렬 조건을 만족해야 합니다.

```
read eight integers: A[1..8]
for each A[i]:
    if A[i] < 1 or A[i] > 8: explode()
    if duplicated(A[i]): explode()

for each A[i]:
    A[i] = 9 - A[i]

for each A[i]:
    selected_node[i] = walk_from_node1(A[i])

relink selected_node list
check node values are in required order
```

노드 값 정리

Node	Value	분석 의미	정렬 판단
node1	208	가장 작은 축에 가까움	후순위
node2	444	중간값	중간
node3	253	작은 편	후순위
node4	404	중간값	중간
node5	759	가장 큰 값	우선
node6	604	큰 값	우선
node7	628	큰 값	우선
node8	747	큰 값	우선
포인터 기반 자료구조를 메모리에서 복원했다.			

6. Case Study - Password Validation 루틴 역분석

학번 기반 password 생성/검증 루틴을 분석하고, XOR 와 byte reorder 구조를 복원한 사례입니다.

항목	정리 내용
분석 목표	student ID 입력값을 기반으로 내부 password 가 생성되고 검증되는 과정을 복원
핵심 함수	validate: 입력 password 비교, shuf: byte/nibble reorder, r: 바이트 순서 변환
핵심 연산	제곱값, XOR 상수, 하위/상위 바이트 추출, 순서 재배열
직무 연결	license check, 환경 체크, anti-analysis 조건, 내부 설정값 복원과 유사한 분석 사고

분석 흐름

- student ID 를 정수로 입력받고 특정 범위에 있는지 먼저 검증합니다.
- 입력된 ID 의 제곱값을 기반으로 상수 XOR 연산을 수행해 password 후보를 생성합니다.
- first password 는 일부 바이트 변환 및 XOR 결과와 비교됩니다.
- second password 는 추가 XOR 와 shuf 함수를 통한 순서 재배열 후 비교됩니다.
- 반대로 주어진 password 로부터 student ID 를 역산하는 과정도 가능하므로, 검증 루틴의 양방향 구조를 이해한 사례입니다.

```
# 단순화한 password validation 흐름
student_id = read_int()
if not valid_range(student_id): exit()

x = student_id * student_id
tmp1 = lower_part(x) ^ CONST_A
first_candidate = reverse_bytes(tmp1 ^ CONST_B)
validate(first_candidate, user_first_password)

tmp2 = upper_part(x) ^ CONST_C
second_candidate = shuf(tmp1 ^ tmp2)
validate(second_candidate, user_second_password)
```

함수 역할 정리

함수/구간	역할	분석 의미
ID validation	입력값이 유효한 범위인지 확인	실행 조건 gate
r function	바이트 순서 뒤집기	값 표현 방식 변환
validate	계산된 password 와 사용자 입력 비교	검증 루틴 핵심
shuf	정해진 순서대로 nibble/byte 재배열	간단한 난독화/순열 변환
검증 알고리즘을 복원하고, 상수/XOR/순서 재배열이 어떤 의미를 갖는지 분석했다		

7. 분석 방법론

- 1 단계: 입력 형식 확인** scanf 포맷 문자열, 문자열 상수, 입력 개수 검증 조건을 먼저 확인했습니다.
- 2 단계: 분기 조건 정리** cmp, test, jmp, jne, je 명령어를 기준으로 폭탄/실패 루틴으로 가는 조건과 통과 조건을 분리했습니다.
- 3 단계: 값 흐름 추적** 입력값이 스택 어느 위치에 저장되는지 확인하고, rax/rdx/rcx 등 레지스터를 거치며 어떻게 변환되는지 추적했습니다.
- 4 단계: 변환 로직 역산** xor, shift, and, table lookup 처럼 입력값을 바꾸는 연산을 역방향으로 계산했습니다.
- 5 단계: 동적 검증** GDB breakpoint, stepi, info registers 를 사용해 분석한 조건이 실제 실행 중에도 맞는지 확인했습니다.

8. 지원 직무와의 연결

- 악성코드 분석은 소스코드가 없는 실행 파일의 행위를 추론해야 하므로, 어셈블리 기반 입력 검증 분석 경험이 기초 역량으로 연결됩니다.
- Bomb Lab 에서 경험한 조건 분기 추적은 악성코드의 anti-debugging, environment check, trigger condition 분석과 유사한 사고방식을 요구합니다.
- XOR, AND, shift, lookup table 분석 경험은 문자열 난독화 해제나 내부 설정값 복원 업무의 기초가 됩니다.
- GDB 를 사용해 레지스터와 스택 상태를 확인한 경험은 동적 분석 도구 사용으로 확장할 수 있습니다.
- 현재는 수업 기반 경험이지만, 리버싱 분석 과정 자체에 흥미를 느꼈고 Windows API, PE 구조, x64dbg/IDA/Ghidra 등으로 학습을 확장하고자 합니다.
-

9. 요약

시스템프로그래밍 수업에서 Bomb Lab 과 검증 바이너리 분석 실습을 수행하며 x86-64 어셈블리 코드의 분기 조건, 스택에 저장된 입력값, 레지스터 값 변화를 추적해 프로그램의 입력 검증 로직을 역분석했습니다. 특히 재귀 함수, XOR/AND 기반 문자열 변환, lookup table, linked list 재배열, password validation 루틴을 분석하면서 컴파일된 실행 파일의 내부 동작을 코드 없이 추론하는 과정에 흥미를 느꼈습니다. 이러한 경험을 바탕으로 악성코드 동적/정적 분석 역량을 더 발전시키고 싶습니다.

Appendix. 원본 분석 노트 일부

아래 이미지는 실제 분석 과정에서 사용한 원본 메모 중 일부입니다. 본문에서는 이를 바탕으로 분석 목표, 방법, 결과를 재구성했습니다.

<func4>

```

0x00005555555568ab <+0>: endbr64
0x00005555555568af <+4>: push %r13
0x00005555555568b1 <+6>: push %r12
0x00005555555568b3 <+8>: push %rbp
0x00005555555568b4 <+9>: push %rbx
0x00005555555568b5 <+10>: sub $0x8,%rsp
0x00005555555568b9 <+14>: movl $0x10(%rdi),%rcx
0x00005555555568c0 <+21>: mov (%rdi,%rcx),%r12d
0x00005555555568c3 <+24>: mov $0x0,%eax
0x00005555555568c8 <+29>: test %r12d,%r12d
0x00005555555568cb <+32>: jne 0x5555555568ff <func4+84>
0x00005555555568cd <+34>: mov %rdi,%rbx
0x00005555555568d0 <+37>: test %rdi,%rdi
0x00005555555568d3 <+40>: mov %rdx,%rbp
0x00005555555568d6 <+43>: mov (%rdi,%rbp),%r12d
0x00005555555568d8 <+45>: cmp (%esi,%eax),%r12d
0x00005555555568da <+47>: je 0x555555556894 <func4+95>
0x00005555555568dc <+49>: mov 0x8(%rdi),%rdi
0x00005555555568de <+53>: test %rdi,%rdi
0x00005555555568e0 <+56>: je 0x5555555568ff <func4+105>
0x00005555555568e5 <+58>: cmpl $0x0,%r12d
0x00005555555568e9 <+62>: je 0x5555555568ff <func4+118>
0x00005555555568eb <+64>: mov 0x10(%rbx),%rdi
0x00005555555568ef <+68>: test %rdi,%rdi
0x00005555555568f2 <+71>: je 0x5555555568ff <func4+79>
0x00005555555568f4 <+73>: cmpl $0x0,%r12d
0x00005555555568f8 <+77>: je 0x5555555568ff <func4+128>
0x00005555555568fa <+79>: mov %r12d,%eax
0x00005555555568fd <+82>: add (%rbx),%eax
0x00005555555568ff <+84>: add $0x8,%rsp
0x0000555555556903 <+88>: pop %rbx
0x0000555555556904 <+89>: pop %rbp
0x0000555555556905 <+90>: pop %r12
0x0000555555556907 <+92>: pop %r13
0x0000555555556909 <+94>: ret
0x000055555555690a <+95>: movl $0x1(%rdx),%r12d
0x0000555555556910 <+101>: mov (%rdi,%r12),%r12d
0x0000555555556912 <+103>: jmp 0x5555555568ff <func4+84>
0x0000555555556914 <+105>: cmpl $0x0,%r12d
0x0000555555556919 <+110>: je 0x5555555568ff <func4+84>
0x000055555555691b <+112>: mov 0x10(%rbx),%rdi
0x000055555555691f <+116>: jmp 0x5555555568ff <func4+73>
0x0000555555556921 <+118>: call 0x5555555568ab <func4>
0x0000555555556926 <+123>: mov %eax,%r12d
0x0000555555556929 <+126>: jmp 0x5555555568eb <func4+64>
0x000055555555692b <+128>: mov %rbx,%rdx
0x000055555555692e <+131>: mov %r13d,%rsi
0x0000555555556931 <+134>: call 0x5555555568ab <func4>
0x0000555555556936 <+139>: add %eax,%r12d
0x0000555555556939 <+142>: jmp 0x5555555568ff <func4+79>

```

그림 1. func4 제귀 함수 분석 메모. 반환값과 분기조건 추적

<phase_5>

```

0x00005555555569c2 <+0>: endbr64
0x00005555555569c6 <+4>: sub $0x28,%rsp
0x00005555555569ca <+8>: mov %fs:0x28,%rax
0x00005555555569d3 <+17>: mov %rax,0x18(%rsp)
0x00005555555569d8 <+22>: xor %eax,%eax
0x00005555555569da <+24>: lea 0xe(%rsp),%rcx
0x00005555555569df <+29>: mov %rsp,%rdx
0x00005555555569e2 <+32>: lea 0x1796(%rip),%rsi
0x00005555555569e9 <+39>: call 0x555555556340 <_iso_c99_sscanf@plt>
0x00005555555569ee <+44>: cmp $0x1,%eax
0x00005555555569f1 <+47>: jle 0x555555556a66 <phase_5+164>
0x00005555555569f3 <+49>: lea 0xe(%rsp),%rdi
0x00005555555569f8 <+54>: call 0x555555556d33 <string_length>
0x00005555555569fd <+59>: cmp $0x0,%eax
0x0000555555556aa0 <+62>: jne 0x555555556a6d <phase_5+171>
0x0000555555556aa2 <+64>: cmpl $0x5,%rsp
0x0000555555556aa6 <+68>: jne 0x555555556a74 <phase_5+178>
0x0000555555556aa8 <+70>: mov (%rsp),%edi
0x0000555555556aa8 <+73>: mov $0x0,%edx
0x0000555555556aa8 <+78>: lea 0xb79(%rip),%rsi
0x0000555555556aa8 <+85>: lea 0xe(%rsp),%rcx
0x0000555555556aa8 <+90>: movl 1(%rdi,%rcx),%eax
0x0000555555556aa8 <+94>: xor %edi,%eax
0x0000555555556aa8 <+96>: and $0xf,%eax
0x0000555555556aa8 <+99>: movzbl (%rsi,%rax,1),%eax
0x0000555555556aa8 <+103>: mov %al,0x7(%rsp,%rdx,1)
0x0000555555556aa8 <+107>: add $0x1,%rdx
0x0000555555556aa8 <+111>: cmp $0x6,%rdx
0x0000555555556aa8 <+115>: jne 0x555555556a1c <phase_5+90>
0x0000555555556aa8 <+117>: movb $0x0,0xd(%rsp)
0x0000555555556aa8 <+122>: lea 0x7(%rsp),%rdi
0x0000555555556aa8 <+127>: lea 0x1649(%rip),%rsi
0x0000555555556aa8 <+134>: call 0x555555556d54 <strings_not_equal>
0x0000555555556aa8 <+139>: test %eax,%eax
0x0000555555556aa8 <+141>: jne 0x555555556a7b <phase_5+185>
0x0000555555556aa8 <+143>: mov 0x18(%rsp),%rax
0x0000555555556aa8 <+148>: sub %fs:0x28,%rax
0x0000555555556aa8 <+157>: jne 0x555555556a82 <phase_5+192>
0x0000555555556aa8 <+159>: add $0x28,%rsp
0x0000555555556aa8 <+163>: ret
0x0000555555556aa8 <+164>: call 0x55555555703c <explode_bomb>
0x0000555555556aa8 <+169>: jmp 0x5555555569f3 <phase_5+49>
0x0000555555556aa8 <+171>: call 0x55555555703c <explode_bomb>
0x0000555555556aa8 <+176>: jmp 0x555555556a02 <phase_5+64>
0x0000555555556aa8 <+178>: call 0x55555555703c <explode_bomb>
0x0000555555556aa8 <+183>: jmp 0x555555556a08 <phase_5+70>
0x0000555555556aa8 <+185>: call 0x55555555703c <explode_bomb>
0x0000555555556aa8 <+190>: jmp 0x555555556a08 <phase_5+143>
0x0000555555556aa8 <+192>: call 0x555555556290 <_stack_chk_fail@plt>

```

그림 2. phase_5의 테이블 기반 문자열 변환 루틴

<phase_6>

```

0x0000555555556aa8 <+0>: endbr64
0x0000555555556aa8 <+4>: push %r15
0x0000555555556aa8 <+6>: push %r14
0x0000555555556aa8 <+8>: push %r13
0x0000555555556aa9 <+10>: push %r12
0x0000555555556aa9 <+12>: push %rbp
0x0000555555556aa9 <+13>: push %rbx
0x0000555555556aa9 <+14>: sub $0x8,%rsp
0x0000555555556aa9 <+21>: mov %fs:0x28,%rax
0x0000555555556aa9 <+30>: mov %rax,0x7(%rsp)
0x0000555555556aa9 <+35>: xor %eax,%eax
0x0000555555556aa9 <+37>: lea 0x10(%rsp),%r15
0x0000555555556aa9 <+42>: mov %r15,0x8(%rsp)
0x0000555555556aa9 <+47>: mov %r15,%rsi
0x0000555555556aa9 <+50>: call 0x55555555703c <readEightNumbers>
0x0000555555556aa9 <+55>: mov %r15,%r13
0x0000555555556aa9 <+58>: mov $0x1,%r14d
0x0000555555556aa9 <+64>: mov %r15,%r12
0x0000555555556aa9 <+67>: jmp 0x555555556bb0 <phase_6+309>
0x0000555555556aa9 <+72>: call 0x55555555703c <explode_bomb>
0x0000555555556aa9 <+77>: cmp $0x7,%r14d
0x0000555555556aa9 <+81>: jle 0x555555556bb8 <phase_6+337>
0x0000555555556aa9 <+87>: jmp 0x555555556bb0 <phase_6+132>
0x0000555555556aa9 <+89>: add $0x1,%rbp
0x0000555555556aa9 <+93>: cmpl $0x7,%ebx
0x0000555555556aa9 <+96>: jmp 0x555555556bb4 <phase_6+301>
0x0000555555556aa9 <+102>: mov (%r12,%rbx,4),%eax
0x0000555555556aa9 <+106>: cmpl %eax,0x0(%rbp)
0x0000555555556aa9 <+109>: jne 0x555555556aa0 <phase_6+89>
0x0000555555556aa9 <+111>: call 0x55555555703c <explode_bomb>
0x0000555555556aa9 <+116>: jmp 0x555555556aa0 <phase_6+89>
0x0000555555556aa9 <+118>: add $0x1,%r14
0x0000555555556aa9 <+122>: cmpl $0x9,%r14
0x0000555555556aa9 <+126>: jne 0x555555556bb9 <phase_6+269>
0x0000555555556aa9 <+132>: mov 0x8(%rsp),%rdx
0x0000555555556aa9 <+137>: add $0x20,%rdx
0x0000555555556aa9 <+141>: mov $0x9,%ecx
0x0000555555556aa9 <+146>: mov %ecx,%eax
0x0000555555556aa9 <+148>: sub 0x0(%r13),%eax
0x0000555555556aa9 <+152>: mov %eax,0x0(%r13)
0x0000555555556aa9 <+156>: add $0x4,%r13
0x0000555555556aa9 <+160>: cmpl %r13,%rdx
0x0000555555556aa9 <+163>: jne 0x555555556bb9 <phase_6+146>
0x0000555555556aa9 <+165>: mov $0x0,%esi
0x0000555555556aa9 <+170>: mov 0x10(%rsp,%rsi,4),%ecx
0x0000555555556aa9 <+174>: mov $0x1,%eax
0x0000555555556aa9 <+179>: lea 0x37ef(%rip),%rdx
0x0000555555556aa9 <+186>: cmpl $0x1,%ecx
0x0000555555556aa9 <+189>: jle 0x555555556bb51 <phase_6+202>
0x0000555555556aa9 <+191>: mov 0x8(%rdx),%rdx
0x0000555555556aa9 <+195>: add $0x1,%eax
0x0000555555556aa9 <+198>: cmpl %ecx,%eax
0x0000555555556aa9 <+200>: jne 0x555555556bb46 <phase_6+191>

```

그림 3. phase_6의 입력 검증 및 node 재배열 분석

```

<validate>
0x00000000000401aa2 <+0>: endbr64
0x00000000000401aa6 <+4>: push %rbp
0x00000000000401aa7 <+5>: mov %rsp,%rbp
0x00000000000401aaa <+8>: sub $0x10,%rsp
0x00000000000401aae <+12>: mov %edi,-0x24(%rbp)
0x00000000000401ab1 <+15>: mov %fs:0x28,%rax
0x00000000000401aba <+24>: mov %rax,-0x8(%rbp)
0x00000000000401abe <+28>: xor %eax,%eax
0x00000000000401ac0 <+30>: movq $0x0,-0x1a(%rbp)
0x00000000000401ac8 <+38>: movb $0x0,-0x12(%rbp)
0x00000000000401acc <+42>: movq $0x0,-0x11(%rbp)
0x00000000000401ad4 <+50>: movb $0x0,-0x9(%rbp)
0x00000000000401ad8 <+54>: nop
0x00000000000401ad9 <+55>: call 0x413e50 <getchar>
0x00000000000401ade <+60>: mov %al,-0x1b(%rbp)
0x00000000000401ae1 <+63>: cmpb %eax,%edi
0x00000000000401ae5 <+67>: je 0x401aed <validate+75>
0x00000000000401ae7 <+69>: cmpb $0xff,-0x1b(%rbp)
0x00000000000401aeb <+73>: jne 0x401aed <validate+55>
0x00000000000401aed <+73>: mov -0x24(%rbp),%edx
0x00000000000401af0 <+78>: lea -0x1a(%rbp),%rax
0x00000000000401af4 <+82>: lea 0x9c535(%rip),%rcx # 0x49e030 "08X" → 16진수 5개
0x00000000000401afb <+89>: mov %rcx,%rsi
0x00000000000401afe <+92>: mov %rax,%rdi
0x00000000000401b01 <+95>: mov $0x0,%eax
0x00000000000401b06 <+100>: call 0x4052c0 <sprintf>
0x00000000000401b0b <+105>: lea 0x9c523(%rip),%rax # 0x49e035 "Please enter the password"
0x00000000000401b12 <+112>: mov %rax,%rdi
0x00000000000401b15 <+115>: mov $0x0,%eax
0x00000000000401b1a <+120>: call 0x4051f0 <printf>
0x00000000000401b1f <+125>: mov 0x8bd2(%rip),%rdx # 0x4ca6f8 <stdin>
0x00000000000401b26 <+132>: lea -0x11(%rbp),%rax
0x00000000000401b2a <+136>: mov $0x9,%esi
0x00000000000401b2f <+141>: mov %rax,%rdi
0x00000000000401b32 <+144>: call 0x413740 <fgets>
0x00000000000401b37 <+149>: lea -0x11(%rbp),%rax
0x00000000000401b3b <+153>: lea -0x1a(%rbp),%rax
0x00000000000401b3f <+157>: mov $0x8,%edx
0x00000000000401b44 <+162>: mov %rcx,%rsi
0x00000000000401b47 <+165>: mov %rax,%rdi
0x00000000000401b4a <+168>: call 0x4010c0 ⇒ strcmp - EAX 2 0 이동
0x00000000000401b4f <+173>: test %eax,%eax
0x00000000000401b51 <+175>: je 0x401b6c <validate+202>
0x00000000000401b53 <+177>: lea 0x9c4fe(%rip),%rax # 0x49e058 "Incorrect. Please check your input."
0x00000000000401b5a <+184>: mov %rax,%rdi
0x00000000000401b5d <+187>: call 0x413ad0 <puts>
0x00000000000401b62 <+192>: mov $0x1,%edi
0x00000000000401b67 <+197>: call 0x404ea0 <exit>
0x00000000000401b6c <+202>: lea 0x9c509(%rip),%rax # 0x49e07c "Correct!"
0x00000000000401b73 <+209>: mov %rax,%rdi
0x00000000000401b76 <+212>: call 0x413ad0 <puts>
0x00000000000401b7b <+217>: mov $0x0,%eax
0x00000000000401b80 <+222>: mov -0x8(%rbp),%rdx
0x00000000000401b84 <+226>: sub %fs:0x28,%rdx
0x00000000000401b8d <+235>: je 0x401b94 <validate+242>
0x00000000000401b8f <+237>: call 0x432ea0 <_stack_chk_fail_local>
0x00000000000401b94 <+242>: leave
0x00000000000401b95 <+243>: ret

```

그림 4. validate 함수 분석 - 입력 password 와 내부 생성값 비교

```

shuf
0x00000000000401a1b <+0>: endbr64
0x00000000000401a1f <+4>: push %rbp
0x00000000000401a20 <+5>: mov %rsp,%rbp
0x00000000000401a23 <+8>: sub $0x20,%rsp
0x00000000000401a27 <+12>: mov %edi,-0x14(%rbp)
0x00000000000401a2a <+15>: mov %esi,-0x18(%rbp)
0x00000000000401a2d <+18>: cmpl $0x8,-0x18(%rbp) → 0 ~ 8
0x00000000000401a31 <+22>: jne 0x401a3a <shuf+31>
0x00000000000401a33 <+24>: mov $0x0,%eax
0x00000000000401a38 <+29>: jmp 0x401aa0 <shuf+133>
0x00000000000401a3a <+31>: mov -0x18(%rbp),%eax → 0
0x00000000000401a3d <+34>: lea 0x1(%rax),%edx
0x00000000000401a40 <+37>: mov -0x14(%rbp),%eax → 비전 2 비트
0x00000000000401a43 <+40>: mov %edx,%esi
0x00000000000401a45 <+42>: mov %eax,%edi
0x00000000000401a47 <+44>: call 0x401a1b <shuf>
0x00000000000401a4c <+49>: mov %eax,-0x14(%rbp) ⇒ 0
0x00000000000401a4f <+52>: mov -0x18(%rbp),%eax → 0 ~ 16진수
0x00000000000401a52 <+55>: shl $0x2,%eax → 7: 111 ⇒ 11100
0x00000000000401a55 <+58>: mov $0xf,%edx
0x00000000000401a5a <+63>: mov %eax,%ecx → 11100 2배 반복
0x00000000000401a5c <+65>: shl %cl,%edx → 1111111111111111
0x00000000000401a5e <+67>: mov %edx,%eax → 1111111111111111
0x00000000000401a60 <+69>: and -0x14(%rbp),%eax → 1111111111111111
0x00000000000401a63 <+72>: mov %eax,%edx
0x00000000000401a65 <+74>: mov -0x18(%rbp),%eax → 1111111111111111
0x00000000000401a68 <+77>: shl $0x2,%eax → 1111111111111111
0x00000000000401a6b <+80>: mov %eax,%ecx
0x00000000000401a6d <+82>: shr %cl,%edx → 1111111111111111
0x00000000000401a6f <+84>: mov %edx,%eax
0x00000000000401a71 <+86>: mov %eax,-0x14(%rbp) → 1111111111111111
0x00000000000401a74 <+89>: mov -0x18(%rbp),%eax → 1111111111111111
0x00000000000401a77 <+92>: cltq
0x00000000000401a79 <+94>: lea 0x0(%rax,4),%rdx → 0
0x00000000000401a81 <+102>: lea 0x9c558(%rip),%rax # 0x4ca0e0 <swap>
0x00000000000401a88 <+109>: mov (%rax,%rax,1),%eax → 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15
0x00000000000401a8b <+112>: shl $0x2,%eax → 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15
0x00000000000401a8e <+115>: mov -0x14(%rbp),%edi
0x00000000000401a91 <+118>: mov %eax,%ecx
0x00000000000401a93 <+120>: shl %cl,%edx → 00000000 00000000
0x00000000000401a95 <+122>: mov %edx,%eax
0x00000000000401a97 <+124>: or -0x8(%rbp),%eax
0x00000000000401a9a <+127>: mov %eax,-0x14(%rbp)
0x00000000000401a9d <+130>: mov -0x14(%rbp),%eax
0x00000000000401aa0 <+133>: leave
0x00000000000401aa1 <+134>: ret

```

그림 5. shuf 함수 분석 - byte reorder 및 비트 이동 로직